

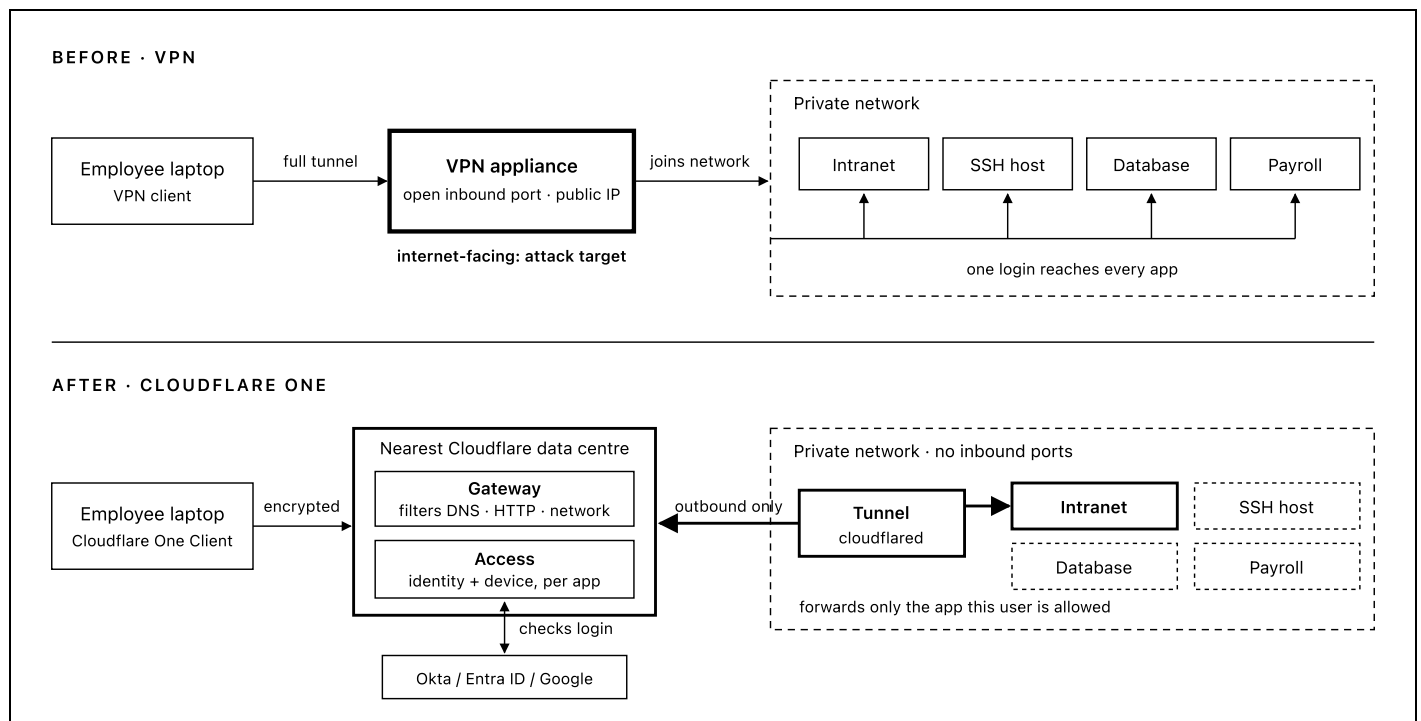
Under the Cloudflare Umbrella

One edge decides who may do what. Staff reach private apps through Cloudflare One instead of a VPN. Shoppers and AI agents reach commerce through a Worker that checks consent, capabilities and mandates against Xano before anything acts. Closed-fee middleware moves data between systems; it does not carry consent, erasure or authority with it.

Yoonsun Lee · Solutions architecture portfolio · September 2026 · [crm-sync.dev](#) · [github.com/persephonepunch](#) · [linkedin.com/in/yoonsunlee](#)

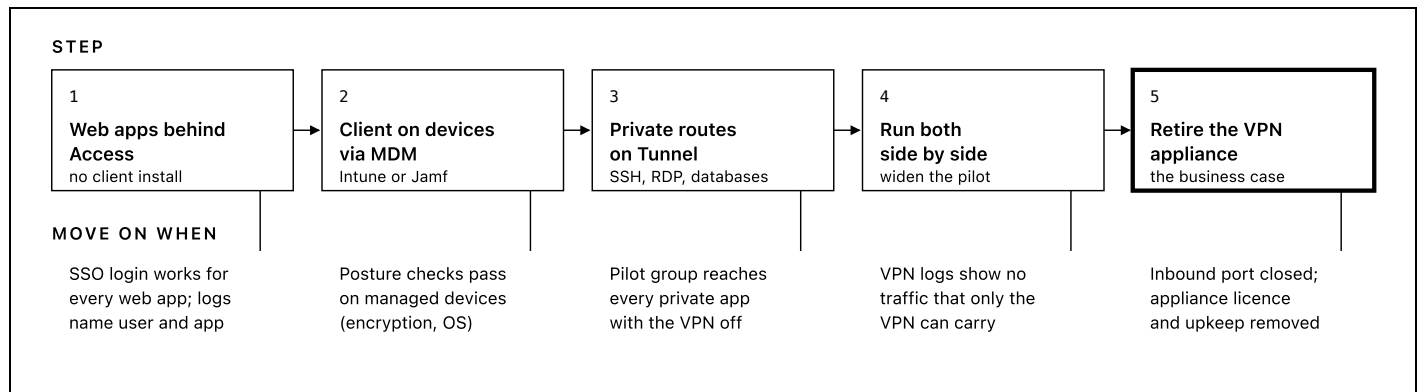
[1 · Staff: VPN to Zero Trust](#) · [2 · Xano Auth and Google ADK](#) · [3 · Middleware and CRM gaps](#) · [4 · Permissions and rendering](#) · [5 · Isolates, containers, VMs](#) · [Least-privilege ladder](#) · [Secrets](#) · [6 · Supply chain](#) · [US build, Korea run](#) · [7 · Shape as code](#) · [8 · ADK vs Xano](#) · [9 · Permissions and lock file](#)

1 · Staff: what changes when the VPN goes



The attack surface and the reach both shrink. The VPN appliance and its open inbound port disappear; the tunnel connects out to Cloudflare, so nothing inside the network listens on the internet. After login, the user reaches only the app their policy names, and each request is checked again. Web-only apps need no client at all: Access alone sits in front of them.

How the migration runs



Each step has a condition to meet before the next one starts, and step 4 is the safety net: the VPN stays up until its own logs show nothing depends on it. Start with the web apps because they need no client and give users the quickest visible change.

Objections to replacing the VPN

"Our VPN works."

It also gives every connected user the whole network, and VPN appliances are among the most attacked devices on the internet. With per-app access, a stolen password opens one app.

"Our legacy apps won't work."

Anything that speaks TCP or UDP runs over the client and the tunnel, and web apps need no client at all. Pilot the hardest app first.

"It will be slower."

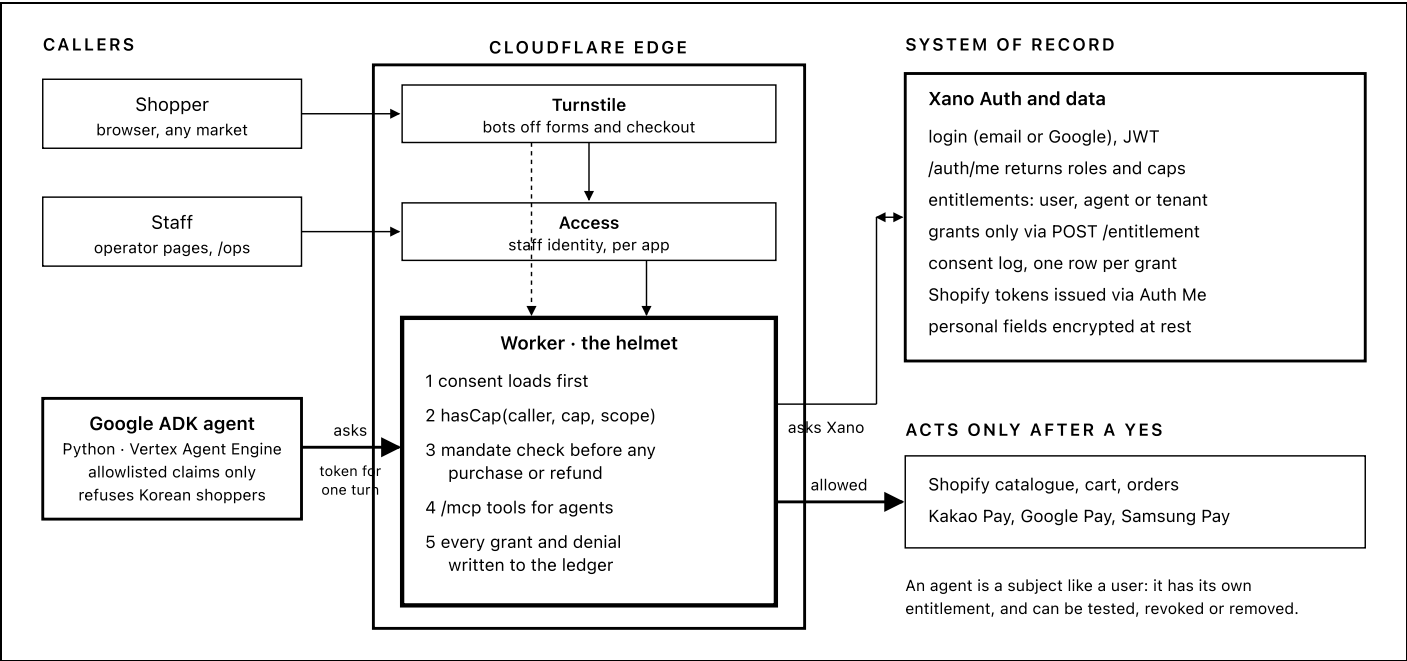
Users connect to the nearest Cloudflare data centre instead of hairpinning through one head office, which matters most for teams spread across a region like APAC.

"What do we show the auditor?"

Every allow and deny is logged per user and per app, which is the evidence SOC 2 access controls (CC6) ask for.

2 · Xano Auth and Google ADK under the umbrella

Every request, from a shopper, a staff member or an AI agent, passes the Cloudflare edge first. The Worker asks Xano who the caller is and what they may do, then acts or refuses. The agent never holds that decision.

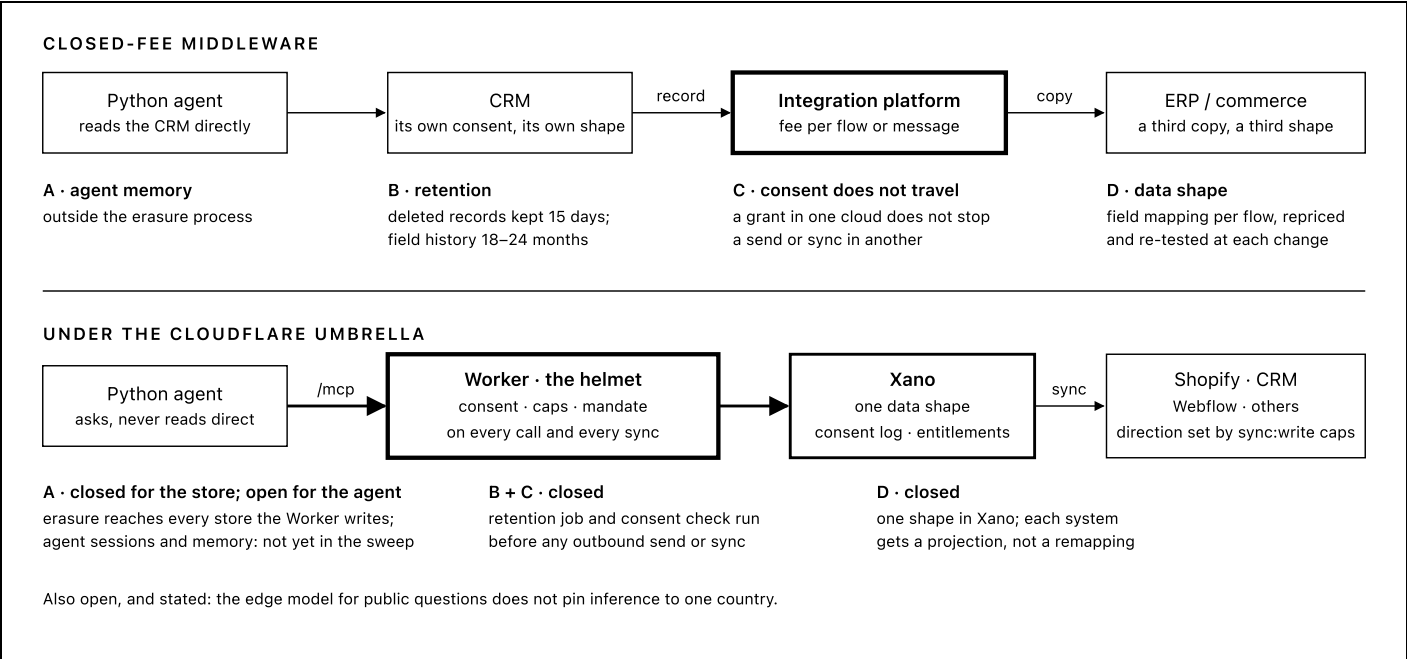


The agent is a shop assistant without the keys: it asks the Worker, the Worker decides with Xano, and the login token is lent for one step and taken back. Dashed line: a shopper's own requests also reach the Worker after Turnstile.

CAPABILITY	WHAT IT ALLOWS	WHO HOLDS IT
crm:agents:read	See registered agents and their activity	Content, audit, QA, security
crm:agents:test	Run an agent against the test gate	Release, QA, security
crm:agents:revoke	Switch an agent off	QA, security
crm:agents:manage	Create agents and change their permissions	Security only
crm:privacy:dsr:read / operate	Review privacy requests / run the erasure	Review: several teams · Operate: security and QA only
crm:keys:ceremony	Rotate a tenant's keys	Security (CISO)
mandate (AP2)	Let an agent buy within a cap, until a date	Granted per shopper, signed, revocable

3 · Closed-fee middleware and CRM: where the gaps are

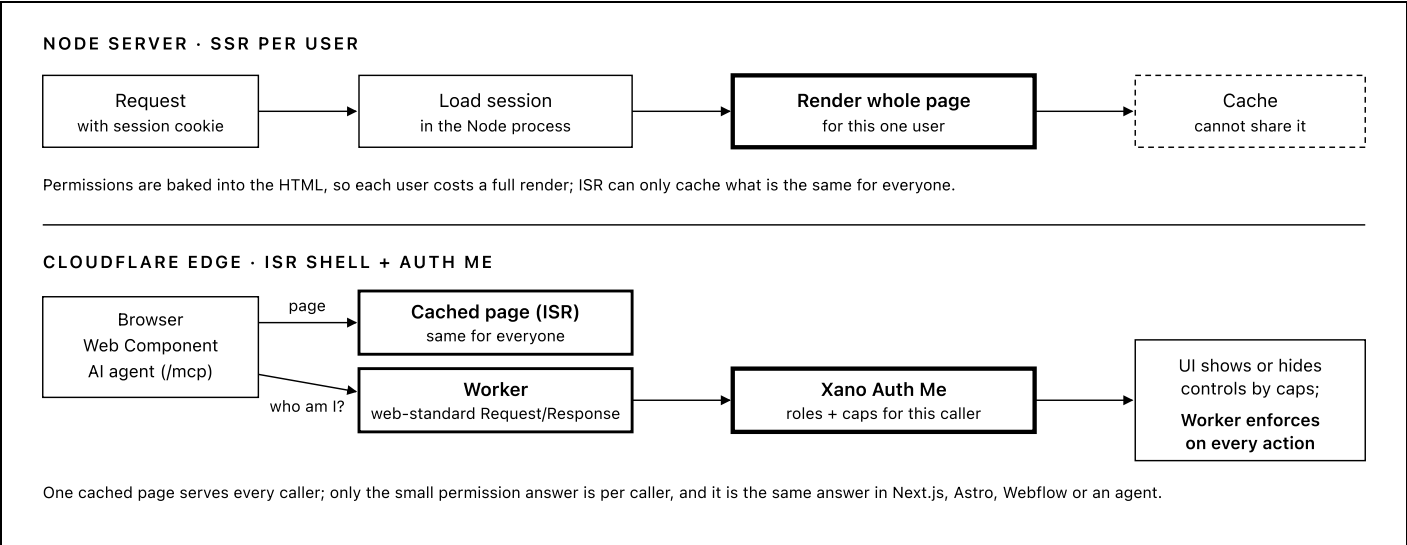
Integration platforms charge to move data between a CRM and commerce systems. They move the record; the consent, retention, erasure and authority around it stay each system's own problem. Under the umbrella, the Python agent and every sync pass one place that carries those rules.



4 · Permissions and rendering: why Auth Me changes SSR

On a Node server, personalising a page usually means rendering it per request (SSR), so nothing can be cached for everyone. With Xano Auth Me returning each caller's roles and capabilities, the page can stay one cached copy (ISR) at the edge, and only the permission answer is per caller. The same answer serves a browser, a Web Component in any framework, and an AI agent through /mcp.

The runtimes line up too. Custom code steps in Xano (Lambdas) run JavaScript or TypeScript on Deno, which uses the same web-standard Request and Response model as Cloudflare Workers. A typical AWS setup runs Node, with its own request objects and packaging. Code written against web standards moves between Xano and the edge with little change; code written for Node usually needs porting.



The rendering choice and the permission check come apart. Hiding a button is a convenience; the Worker's check on the action is the control, so a cached page never grants anything.

	NODE SERVER, SSR PER USER	EDGE ISR + XANO AUTH ME
Where permissions live	Baked into each rendered page	One Auth Me answer per caller: roles and capabilities
Caching	Personalised pages cannot be shared	The page is cached once and served to everyone
Who can use the answer	The one framework that rendered it	Browsers, Web Components in any framework, AI agents
Where the check is enforced	In the render, per framework	In the Worker, on every action, for every caller
Runtime model	A long-running Node process with its own request objects	Web-standard Request and Response in isolates, the same model Deno uses
Custom code in the backend	AWS Lambda functions, typically on Node	Xano Lambda steps: JavaScript or TypeScript on Deno; XanoScript for everything else

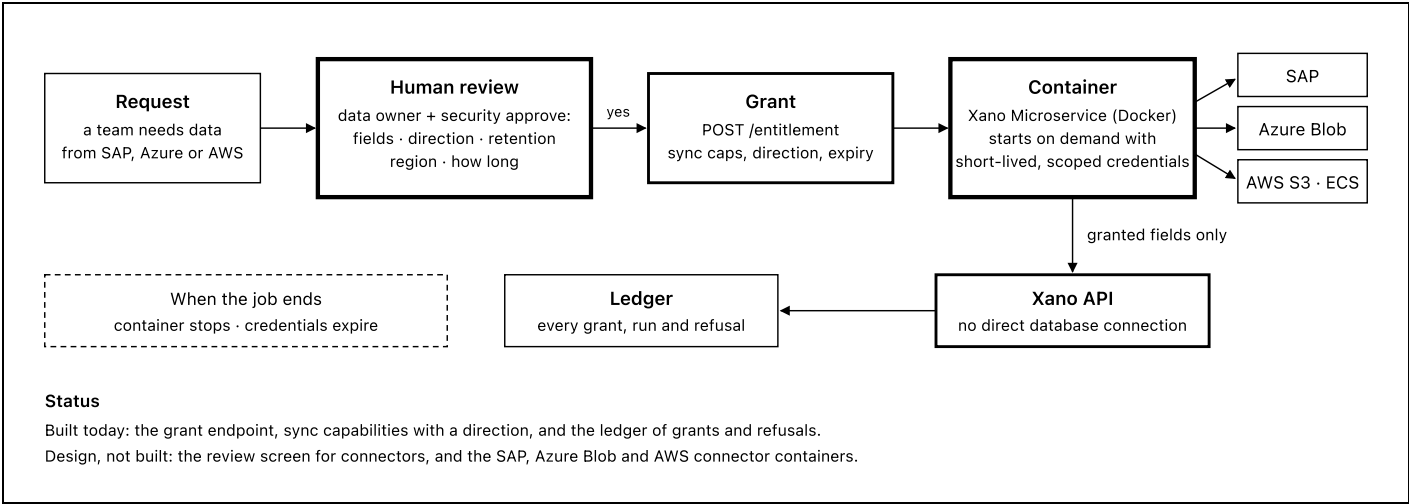
5 • Isolates, containers, VMs: separation as a control

Each layer runs code with less reach than the one below it, and nothing moves down a layer unless the job needs more control. Separation is the security benefit: a fault or a compromise stays inside the layer where it happened.

LAYER	WHERE IT RUNS	STARTS	WHAT IT CAN REACH	IF IT IS COMPROMISED
Cloudflare Worker V8 isolate	Cloudflare edge, near the caller	Milliseconds, per request	Only its declared bindings (KV, R2, D1, secrets) and outbound fetch	One request's memory. No operating system, no disk
XanoScript managed runtime	The Xano instance	Per request	Xano's database and cache, through the language only	No process or operating-system access to abuse
Xano Lambda Deno sandbox	Inside a function stack	Per call, with a timeout	What the stack passes in, including \$env	Ends at the timeout, but can read environment values: keep secrets out of any Lambda that handles untrusted input
Xano Microservice Docker container	Xano's Kubernetes, in the instance's region	On demand	What its image and credentials allow; Xano data only through the Xano API	The container and its credentials. Not the database: there is no direct database connection to steal
Cloud VM	A cloud provider	Minutes	A full operating system and network	The whole machine, which you patch and monitor yourself

On-demand connectors to SAP, Azure and AWS, after human review

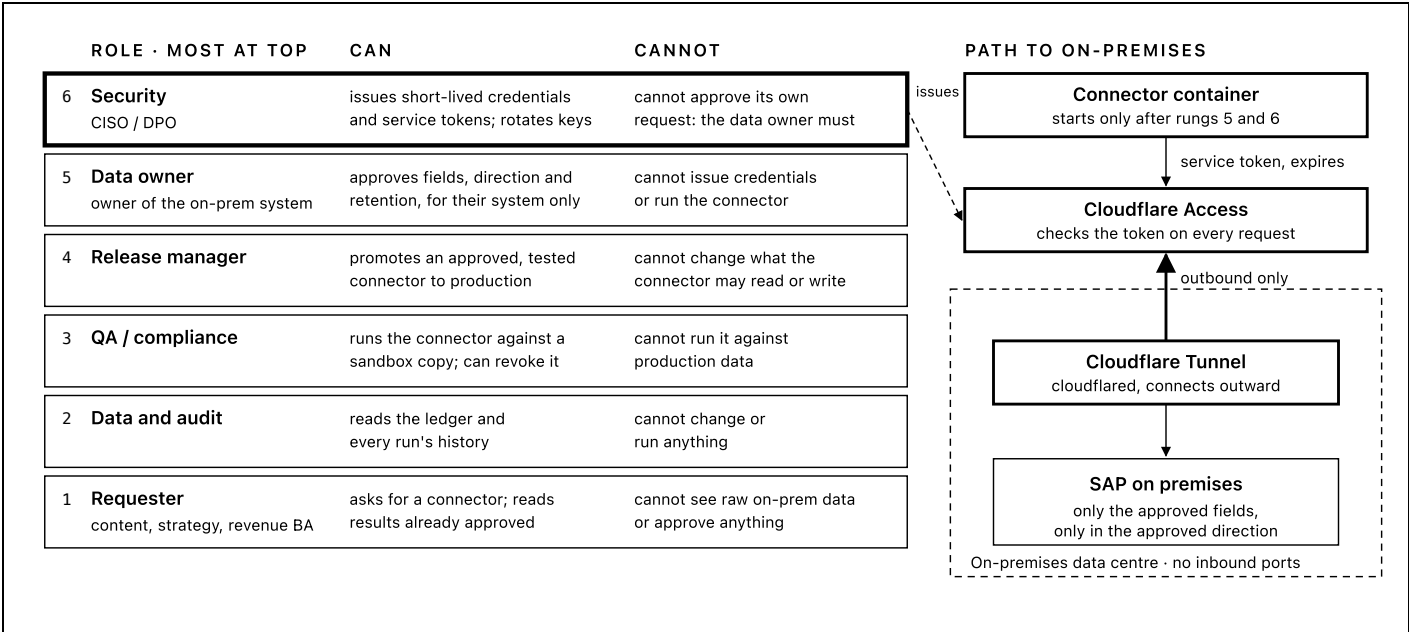
A connector to an enterprise system runs as a container that does not exist until a person has approved exactly what it may read and write. It starts with short-lived credentials scoped to that approval, reaches Xano only through the API, and stops when the job is done.



No standing access to the enterprise system exists between jobs. The approval, not the container, holds the authority, and it expires. A compromised connector can reach only the fields it was granted, and only until its credentials run out.

Human stakeholders: the least-privilege ladder to on-premises systems

Each person holds only the rung their job needs. The same request needs three different people: one asks, the data owner approves, security issues the credentials. No one can do all three, so no single account, stolen or misused, can open an on-premises system on its own.



Least privilege for people, not only for code. Rungs 2, 3 and 6 map to capabilities that exist today (reading agents and the ledger, testing and revoking, key ceremonies); the connector rungs and the on-premises path are design. The on-premises site opens no firewall port: the Tunnel connects out to Cloudflare, and Access checks the connector's token on every request, exactly as it does for staff in section 1.

Secrets along the escalation

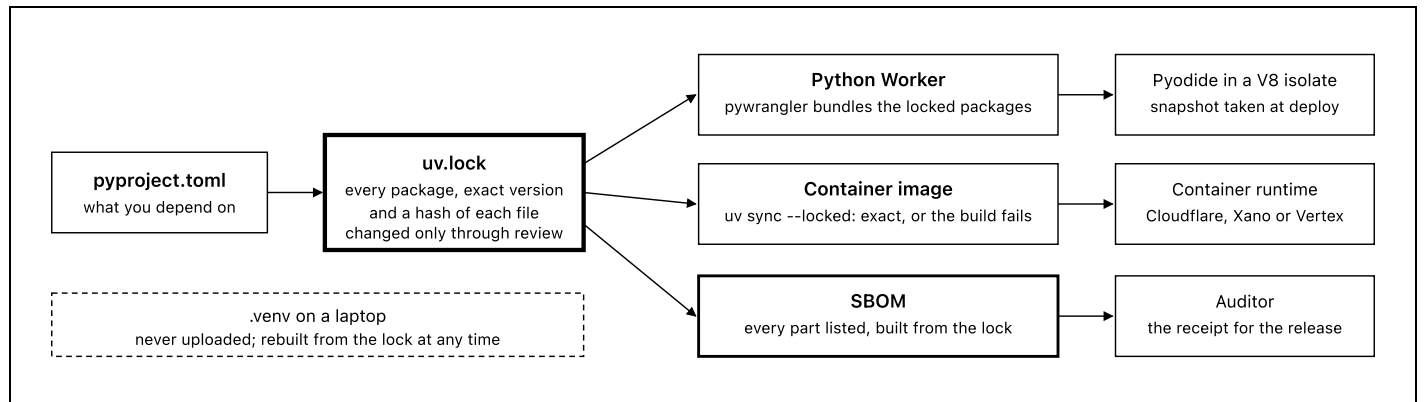
The ladder decides who may act; these rules decide where a secret may exist while they do. A secret only ever lives in a secret store, reaches a process at the moment it starts, and expires with the job. Plain-text files on a laptop are the weakest link, so they hold configuration and nothing else.

WHERE A SECRET COULD SIT	RULE
.env and other text files	Configuration only: URLs, project and region names, IDs. Never a key or token. Owner-only permissions (600), never committed.
.venv (Python packages)	Dependencies only. Never holds a secret, never committed, and rebuilt from the lock file rather than copied.
Developer sign-in files	Cloud sign-in tokens on a laptop are real credentials: owner-only, short sessions, and revoked when the person leaves the ladder.
Container images	No secret is ever baked into an image. The connector's credentials are injected when it starts, from the secret store, and expire when it stops.
Cloudflare Workers	Encrypted secrets, typed straight into the platform at a prompt. Never pasted into chat, email or a ticket.
Xano	Environment variables in the workspace; functions read them, people do not copy them out.
The key register and the ledger	Record each key by fingerprint (the first characters of its hash), who rotated it and when. Never the value.
Each rung of the ladder	Only Security (rung 6) handles secret values, and only to put them into a store. Every other rung sees a fingerprint, a status or nothing.

6 · Supply chain: the lock file, and who may publish

What runs in production should be exactly what was reviewed. The lock file fixes which code goes in; the publishing rule fixes who can put it live. An outside team can write, test and stage everything, but it never holds a credential that publishes to production.

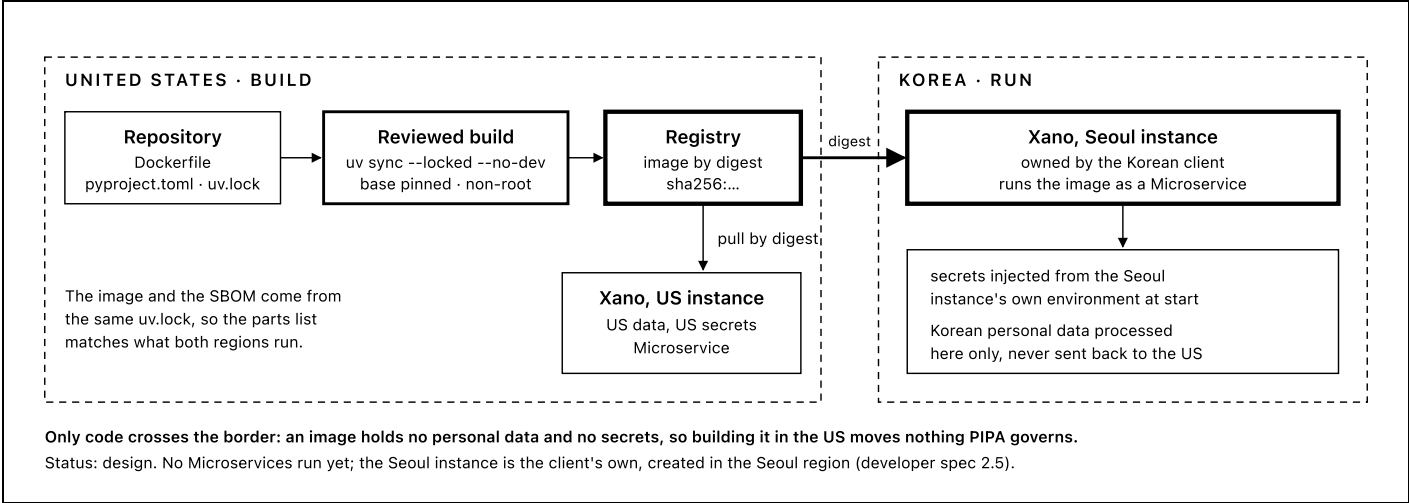
The lock file



Python Workers accept pure-Python packages and packages built for Pyodide; code with native libraries, such as Google's ADK and its gRPC dependency, runs as a container built from the same lock file. A package swapped upstream fails its hash and does not install.

Build once in the US, run in Korea: the Xano image flow

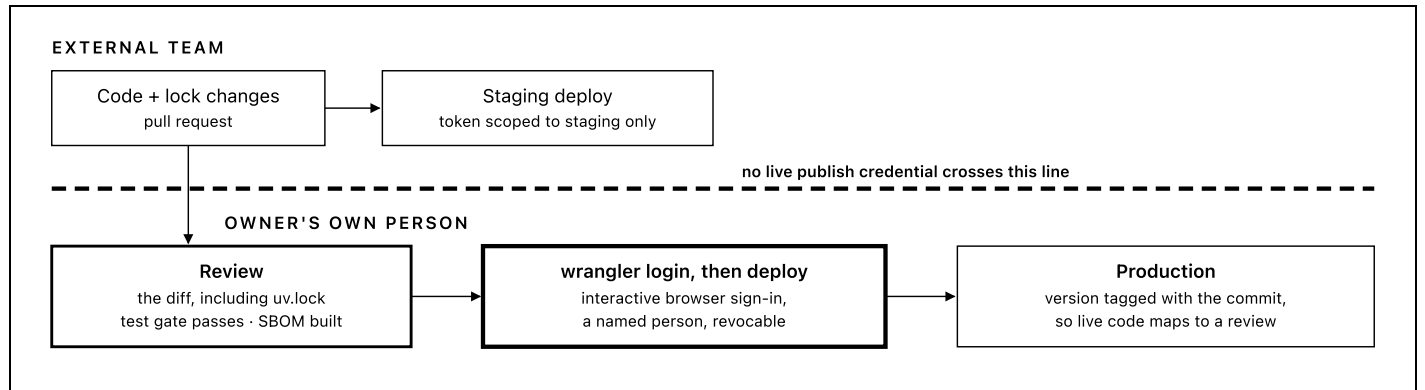
Xano does not build containers; it runs a finished image pulled from a registry. The Dockerfile and lock file live in your repository, the image is built once in your reviewed pipeline, and each Xano instance pulls that exact image by digest. The image holds code only, so it can be built and stored in the US while Korean personal data never leaves the Seoul instance.



One reviewed build, two regions, identical code, proven by the digest. Xano pulls only from well-known registries, and private-registry credentials carry an expiry: give each instance its own short-lived pull credential. Pin the digest in Xano, never a tag such as :latest, so a re-pushed tag cannot change what Korea runs.

Interactive publishing: no external team holds live credentials

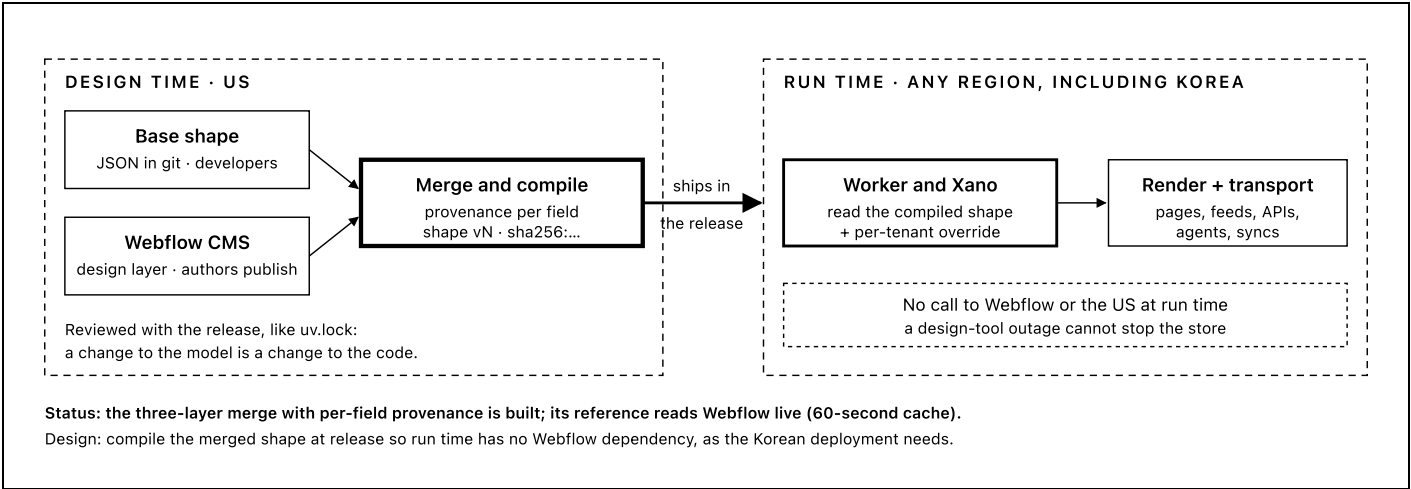
External developers work in their own development account or the staging environment, with a token that can reach staging and nothing else. Production is published by the owner's own person, signed in to Wrangler interactively in a browser, from the reviewed commit. The sign-in belongs to a named person, can be revoked with their account, and is never handed over as a token.



If an agency's laptop or CI is compromised, the attacker reaches staging, not customers. Built today: the test gate that blocks a deploy, commit-tagged versions, the SBOM from the lock, and the handover rule that production keys are entered by the owner, not the developer. The staging-only token and interactive-only production publishing are the rule to set for each client.

7 · Shape: designed in the US, shipped as code, no design-tool dependency at run time

A shape is the data model a design surface authors and the runtime renders: fields, where each value came from, and a version. It is designed once (model, view and controller agreed), then compiled into a versioned file that ships in the release like the lock file. From then on the data design is code: day-to-day compute and transport read the compiled shape, and nothing at run time calls Webflow or reaches back to the US.



Model, view and controller are settled at design time, once. After that the shape travels with the code: reviewed, hashed, versioned and rolled back like any release. Authors keep using Webflow to change the design; their change reaches customers at the next release, not the next request.

8 • Where it runs: Google ADK compared with the Xano function stack

Both tools let you build logic in a visual or local environment and run it somewhere else. The difference is how visible and how fixed the location is. In Xano the region belongs to the instance and is shown in the editor; in ADK it is a setting you choose at each step, and the local development server runs wherever you start it.

	GOOGLE ADK • ADK WEB AND AGENT ENGINE	XANO FUNCTION STACK • CANVAS, STACK, XANOSCRIPT
What it is	A Python agent framework. <code>adk web</code> is its local development server and test UI	A hosted backend. Canvas, stack and XanoScript are three views of the same function
Where the builder runs	On the machine that starts it, for example a laptop in New York	Hosted, on the instance; the editor's status bar shows its region (for example United States)
Where production runs	Agent Engine, in the region chosen at deploy: <code>asia-northeast3</code> for Seoul	The instance's region. Korea needs a Seoul instance, owned by the client
Where the model runs	The Vertex AI region in <code>GOOGLE_CLOUD_LOCATION</code> . Seoul runs <code>gemini-2.5-flash</code> ; its shared capacity rejected multi-call turns in testing	Only where a function calls a model; the provider it calls decides the location
Where state lives	<code>adk web</code> : in memory on the laptop. Agent Engine: sessions and Memory Bank in the deploy region	The instance's database and cache, in its region
Where secrets live	Local <code>.env</code> holds configuration only; sign-in comes from the developer's Google login. Production uses the Agent Engine configuration	Keys and Variables, read inside the function as <code>\$env</code> , never returned in a response
How a request is refused	The agent cannot refuse on authority: it asks the Worker, and the Worker checks capabilities and mandates	A precondition step stops the request with a real HTTP error. (<code>A throw</code> answers 200, so use a precondition for refusals)
Versions	Git and <code>uv.lock</code> ; a new version is a redeploy	Branch (for example <code>v1</code>) and data source (live or test), switched in the editor
Serves Korean shoppers today	No. It refuses market KR and hands Korean shoppers to the Worker's chatbot on Workers AI	Only from the client's Seoul instance; today's instance is in the United States

The rule for Korea is the same for both: the model, the state and the data layer all have to be in Korea before Korean personal data is served. Running `adk web` against the Seoul model is not enough, because its tools still reach a data layer in the United States.

9 • Row permissions and the lock file: both, not either

Permissions stored as rows scale with every user, agent and tenant, and the lock file covers the whole build at once. Neither is more secure than the other: they stop different attacks, and each fails without the other. Row permissions are enforced by your code; the lock file is what makes that code trustworthy.

	ROW-BASED PERMISSIONS (ENTITLEMENTS)	THE LOCK FILE (UV.LOCK)
Question it answers	Who may do what, to which data?	Is the code running what we reviewed?
Kind of control	Authorisation, checked at run time	Supply-chain integrity, fixed at build time
Granularity	Per subject (user, agent, tenant), per capability, per scope, on every request	One rule for the whole build: exact versions and file hashes
Scales with	People, agents and tenants: add a row, not code	Dependencies: every package pinned
Stops	A caller doing more than it was granted: another tenant's data, a purchase without a mandate	A tampered or silently upgraded package reaching production
Does not stop	Malicious code inside your own process	A person or agent misusing access it legitimately holds

Why both are needed. A compromised package runs as your code, with your code's access: it can read environment secrets, call the data layer with the service's own credentials, and skip the permission check entirely. And a perfect lock file does nothing about a person who was granted too much. Blanket coverage is right for the lock file, because one bad package anywhere compromises the whole process; fine-grained control is right for permissions, because that is where people and agents differ.

How the layers stack on this page.

1. The lock file and the review gate: the right code is running (section 6).
2. Runtime separation: if code is compromised anyway, the damage stays in its layer (section 5).
3. Row permissions, mandates and the ledger: the right code lets each caller do only what it was granted, and records it (section 2).
4. The human ladder and interactive publishing: the right people control changes to layers 1 and 3 (sections 5 and 6).

The lock file makes sure the code is honest; the permissions make sure the callers are.

A VPN trusts the network, and middleware trusts the record. Under the Cloudflare umbrella nothing is trusted by default: the person, the device, the agent and the sync are each checked on every request, against one system of record.

Architecture from the CRM Sync estate: the Google ADK concierge is built and tested, not yet deployed. Product names as in Cloudflare's documentation, September 2026: the device software is the Cloudflare One Client (formerly WARP); site-to-site links run on Cloudflare WAN.